

Día D — qué ejecuta cada uno

Misma constitución que las cinco fases: **una sola persona escribe la especificación y reparte los tickets, el resto solo los ejecuta.** Ramas y archivos ya están decididos y no se inventan en vivo. Lo único que se escribe a mano es **la Historia que dicte el profesor.**

Quién	Su rama	Su ticket	Sus archivos de código
Sergio	main	escribe <code>spec.md</code> y los tres <code>hu-viva/*.md</code>	ninguno — especifica y reparte
Mateo	feature/hu-viva-ui	<code>specs/001-ai-media-generator/hu-viva/mateo.md</code>	<code>app/**/*.jsx</code> , <code>components/**</code>
Johan	feature/hu-viva-api	<code>specs/001-ai-media-generator/hu-viva/johan.md</code>	<code>app/api/**/*.route.js</code>
Tomás	feature/hu-viva-datos	<code>specs/001-ai-media-generator/hu-viva/tomas.md</code>	<code>lib/**</code> , <code>test/**</code>
Santiago	main	ninguno	merge, build y despliegue

Igual que en `tasks.md`: ningún archivo de código aparece en dos filas, y por eso las tres ramas corren a la vez. Mateo, Johan y Tomás **no** escriben specs — abren su ticket y lo implementan.

Paso 0 — 20 minutos antes, los cinco

```
cd ~/Documents/web/speckit-ai-generator
git checkout main
git pull origin main
npm install
npm test # debe decir 0 failed
```

Santiago, además:

```
npm run smoke https://ia-generator-
openspec.vercel.app
```

Deja el panel de Vercel abierto y la URL cargada en el celular.

Paso 1 — Sergio especifica y reparte · ≈ 5 min · rama `main`

Es la Fase 1 de siempre: **bloquea a todos**. Nadie crea su rama hasta que este push termine. Arranca en `main` al día — `git checkout main && git pull origin main`.

1.1 — Registra la Historia. Pega esto en el agente, con las historias literales del profesor:

```
/speckit-specify Agrega dos User Stories nuevas al final de la seccion User Scenarios de
specs/001-ai-media-generator/spec.md, numeradas User Story 5 y User Story 6.
HU 5: "historia literal que dictó el profesor"
HU 6: "segunda historia literal"
Cada una con sus Acceptance Scenarios numerados en formato Given/When/Then.
NO crees una feature nueva. NO corras create-new-feature.sh. NO cambies de rama.
NO toques plan.md ni tasks.md. Solo edita ese spec.md que ya existe.
```

1.2 — Reparte los tickets. Mismo corte de propiedad que las Fases 2, 3 y 4:

```
/speckit-tasks Escribe los tres tickets de la HU en vivo en
specs/001-ai-media-generator/hu-viva/: mateo.md, johan.md y tomas.md. Solo esos tres archivos.
```

Reparte las User Stories 5 y 6 de spec.md con el mismo corte de propiedad que las Fases 2, 3 y 4 de tasks.md: mateo -> `app/**/*.jsx` y `components/**`, johan -> `app/api/**/*.route.js`, tomas -> `lib/**` y `test/**`.

Cada archivo lleva cuatro secciones:

1. Historia — la parte de la HU que le toca, textual desde spec.md.
2. Criterios de Aceptacion — numerados CA-1, CA-2, CA-3, en formato Dado / Cuando / Entonces.
3. Plan — la lista exacta de archivos que va a crear o modificar, y por que cada uno.
4. Tasks — numeradas, y cada una dice que CA cubre.

Reglas: ningún archivo de código puede aparecer en dos secciones Plan. No toques plan.md ni tasks.md. Si la Historia guarda datos, van a MongoDB por `lib/db.js`, nunca en memoria. Si el reparto no cierra, parate y dimelo antes de escribir.

1.3 — Sube las cuatro cosas. Hasta que esto no esté arriba, los demás no tienen ticket. Cuando el push termine, dilo en voz alta:

```
git add specs/001-ai-media-generator/spec.md specs/001-ai-media-generator/hu-viva/
git commit -m "HU en vivo: User Stories 5 y 6, y los tres tickets repartidos"
git push origin main
```

Paso 2 — Mateo, Johan y Tomás, cada uno en su máquina · ≈ 5 min

2.1 — Crea tu rama, solo cuando Sergio haya dicho que ya está arriba. El `pull` es lo que te trae tu ticket. Copia solo tu columna:

MATEO

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
ui
```

JOHAN

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
api
```

TOMÁS

```
git checkout main
git pull origin main
git checkout -b feature/hu-viva-
datos
```

2.2 — Abre tu ticket. Ya está escrito y ya está en tu rama. Cambia `mateo` por `johan` o `tomas` según quién seas:

```
cat specs/001-ai-media-generator/hu-viva/mateo.md
```

Léelo entero antes de pedirle nada al agente. **Plan** dice exactamente qué archivos puedes tocar; **Tasks** dice qué CA cubre cada una. Si algo no cuadra, díselo a Sergio ahora — **tú no editas el ticket**.

2.3 — Implementa `test-first` (misma sustitución de nombre):

```
Implementa specs/001-ai-media-generator/hu-viva/mateo.md, test-first:
por cada Criterio de Aceptacion, primero la prueba que falla y despues el codigo que la pasa.
```

```
Nombra cada prueba empezando por su criterio: it("CA-2: ...").
Toca unicamente los archivos de la seccion Plan de ese documento. Ninguno mas.
NO edites ese .md, ni spec.md, ni plan.md, ni tasks.md: el ticket ya esta cerrado.
Si un Criterio de Aceptacion esta mal o te falta un archivo, parate y dimelo.
Al terminar corre npm test y dejalo en verde.
```

2.4 — Prueba, `commit` y `push`. Copia solo tu columna:

MATEO

```
npm test
git add -A
git commit -m "feat: HU viva UI
(CA-1 a CA-3)"
git push -u origin feature/hu-
viva-ui
```

JOHAN

```
npm test
git add -A
git commit -m "feat: HU viva API
(CA-1 a CA-3)"
git push -u origin feature/hu-
viva-api
```

TOMÁS

```
npm test
git add -A
git commit -m "feat: HU viva
datos (CA-1 a CA-3)"
git push -u origin feature/hu-
viva-datos
```

Si `npm test` **sale rojo, no hagas push**. Dile al agente cuál CA falla, que lo arregle y vuelva a correr `npm test`. Cuando tu push termine, dilo en voz alta.

Paso 3 — Santiago integra y despliega · ≈ 2 min · rama `main`

Solo cuando los tres hayan dicho que subieron:

```
git checkout main
git pull origin main
git fetch origin
```

Un merge por línea. Espera a que termine cada uno:

```
git merge --no-ff origin/feature/hu-viva-ui
git merge --no-ff origin/feature/hu-viva-api
git merge --no-ff origin/feature/hu-viva-datos
```

Si un merge se enreda: `git merge --abort` y reintenta solo esa rama.

Con los tres adentro:

```
npm test
npm run build
git push origin main
```

Ese push despliega en Vercel solo. Mientras construye, proyecta el grafo:

```
git log --graph --oneline -15
```

Paso 4 — Todos validan sobre el despliegue · ≈ 1 min

1. **Santiago** espera el check verde del deploy en Vercel (1–2 min) y corre:

```
npm run smoke https://ia-generator-openspec.vercel.app
```

2. Todos abren `https://ia-generator-openspec.vercel.app` en el celular, con datos móviles, y recorren la Historia nueva de punta a punta.

3. Si preguntan por la trazabilidad, el dueño abre su `hu-viva/nombre.md`, señala el CA y corre esa sola prueba:

```
npm test -- -t "CA-2"
```